

Polynomial Regression

with Scikit-Learn — Regression Task

Build, evaluate, and analyze polynomial regression models using real datasets — and understand the metrics that tell you how good your model actually is.

Prepared by

Omar Ashraf

Vice Head of AI Sub-Team • IEEE SSCS

Task Overview

You've learned the tools — now it's time to use them. In this task, you'll work with real datasets the way a machine learning engineer would: fit polynomial models, measure their performance honestly, and diagnose what's going wrong when the numbers don't look right. A model that scores perfectly on training data but fails on new data is useless — your job is to find the right balance.

Your Dataset

- **UCI Auto MPG dataset** — predict MPG (miles per gallon) from other features like horsepower.
(`pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data')`)
- **Custom dataset** — generate synthetic data with `np.random` or find an interesting regression dataset on Kaggle.

Task Sections

Step

1

Data Loading & Exploration

💡 Start here. Load your dataset, inspect its shape, and run basic statistics. Plot the target variable's distribution. Are there outliers? Is the target skewed?

NOTE: this step is important, it saves you hours of debugging later.

Step

2

Linear Baseline

💡 Fit a simple `LinearRegression` model first. Compute MSE, MAE, RMSE, MAPE, and R^2 on the test set. This baseline is your benchmark — every polynomial model you build must be compared against it.

NOTE: a baseline model helps you determine if the more complex model actually learned something

Step

3

Polynomial Feature Engineering

💡 Use `PolynomialFeatures` to expand your input space. Fit models at degree 2, 3, 4, and 5. Plot training vs. validation RMSE and R^2 as degree increases. Identify exactly where the model starts overfitting.

Step

4

Regularization (Ridge & Lasso)

💡 Apply Ridge (L2) and Lasso (L1) regularization to your best polynomial degree. Use cross-validated `GridSearchCV` to tune the alpha hyperparameter. Report the best alpha and compare regularized vs. unregularized test metrics.

Step

5

Metrics Deep Dive

💡 For your best model, compute and report all five metrics: MSE, MAE, RMSE, MAPE, and R^2 . Build a comparison table across all your models (linear baseline, poly degree 2–5, Ridge, Lasso). Discuss what each metric is telling you.

Step

6

Residual Analysis

💡 Plot residuals ($\hat{y} - y$) vs. fitted values for your best model. Are residuals randomly scattered around zero? Any patterns? A funnel shape means heteroscedasticity — your model is missing structure. Explain what you see.

Step

7

Insight & Conclusions

💡 What did the numbers reveal? For each dataset, write 2–3 clear observations. Which metric mattered most for this problem and why? What degree or regularization worked best, and what does that tell you about the data's complexity?

A note on datasets

You can use any of the listed datasets for any step, as long as the model and metrics are meaningful and your analysis tells a clear story about the data. Quality of insight matters more than quantity of code.

Submission

Your submission must include:

Deliverable	Description
Python Code	A clean, well-commented Notebook (.ipynb) implementing polynomial regression and computing all five metrics on at least one datasets.
Written Report	A PDF or Word document discussing your findings, methodology, and conclusions. Include a brief explanation of Ridge & Lasso regularization and why they matter in polynomial models, and discuss the differences between MSE, MAE, RMSE, MAPE, and R^2 — when to use each and what their limitations are.

Note: you should upload the code and report to **Github**, and send the Github repo link in the submission.

A Note on Grading

The quality of your analysis matters more than the quantity of code. Focus on choosing the right polynomial degree, honestly comparing your metrics, and diagnosing what the residual plot is telling you. A well-reasoned explanation of one model beats five models with no commentary.

Good luck! — Omar Ashraf | Vice Head of AI Sub-Team | IEEE SSCS